HDOC Day-53

Maximum Subarray Sum Using Functions and Pointers in C

Aim

To write a C program using a variable-length integer array, user-defined functions, pointers, and nested loops to find the maximum sum among all contiguous subarrays of size k. Display -1 if k is invalid.

Algorithm

1. Start the program and read the array size.
2. Check that the array size is positive; otherwise display an error and stop.
3. Declare a variable-length array and read its elements.
4. Read k, the required length of each contiguous subarray.
5. Pass the array, size, and k to max_subarray_sum() using a pointer.
6. If k is less than or equal to zero or exceeds size, display -1 and return.
7. For every valid starting index, calculate the sum of k consecutive elements using an inner loop.
8. Display each subarray and its sum, and update the maximum sum.
9. Display the maximum subarray sum and stop.

C Program

```c
#include <stdio.h>

void max_subarray_sum(int *arr, int size, int k);

int main()
{
    int size, k, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d array elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter the value of k: ");
    scanf("%d", &k);

    max_subarray_sum(arr, size, k);
    return 0;
}

/* Function to find maximum subarray sum */
void max_subarray_sum(int *arr, int size, int k)
{
    int i, j;
    long long sum, maxSum = 0;
    int firstSubarray = 1;

    if (k <= 0 || k > size)
    {
        printf("Maximum Subarray Sum: -1\n");
        return;
    }

    printf("\nContiguous subarrays of size %d:\n", k);

    for (i = 0; i <= size - k; i++)
    {
        sum = 0;
        printf("Subarray: ");

        for (j = i; j < i + k; j++)
        {
            printf("%d ", *(arr + j));
            sum += *(arr + j);
        }

        printf(" | Sum = %lld\n", sum);

        if (firstSubarray || sum > maxSum)
        {
            maxSum = sum;
            firstSubarray = 0;
        }
    }
```

```
    printf("\nMaximum Subarray Sum: %lld\n", maxSum);
}
```

Explanation

The array is passed by its address using `max_subarray_sum(arr, size, k)`. Inside the function, `*(arr + j)` accesses the element at index j. The outer loop chooses each contiguous subarray, and the inner loop calculates its sum.

Worked Example

For arr = [2, 1, 5, 1, 3, 2] and k = 3, the contiguous subarray sums are 8, 7, 9, and 6. Therefore, the maximum subarray sum is **9**.

Test Case Table

Test	Input Array	k	Expected Result
1	2 1 5 1 3 2	3	9
2	1 2 3 4 5	2	9
3	5 4 3 2 1	3	12
4	-5 -2 -3 -1	2	-4
5	10 20 30 40	4	100
6	4 2 7 1 9	1	9
7	1 2 3 4 5	0	-1
8	1 2 3 4 5	6	-1
9	size = 0	-	Invalid array size

Sample Outputs

Test Case 1: Maximum Subarray Sum Exists

```
Enter the size of array: 6
Enter 6 array elements: 2 1 5 1 3 2
Enter the value of k: 3

Contiguous subarrays of size 3:
Subarray: 2 1 5  | Sum = 8
Subarray: 1 5 1  | Sum = 7
Subarray: 5 1 3  | Sum = 9
Subarray: 1 3 2  | Sum = 6

Maximum Subarray Sum: 9
```

Test Case 2: Negative Elements

```
Enter the size of array: 4
Enter 4 array elements: -5 -2 -3 -1
Enter the value of k: 2

Contiguous subarrays of size 2:
Subarray: -5 -2  | Sum = -7
Subarray: -2 -3  | Sum = -5
Subarray: -3 -1  | Sum = -4

Maximum Subarray Sum: -4
```

Test Case 3: Invalid k

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5
Enter the value of k: 6
Maximum Subarray Sum: -1
```

Test Case 4: Invalid Array Size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program uses a variable-length array, pointers, a user-defined function, nested loops, and conditional statements to identify all contiguous subarrays of size k and find their maximum sum. It handles positive and negative values and displays -1 when k is invalid.